

A NEAR-LINEAR-TIME SOLVER FOR BILEVEL CONGESTION NETWORK DESIGN VIA DIRECTED NONLINEAR LAPLACIAN FLOW*

OREN E. LIVNE†

Abstract. Bilevel traffic network design—optimizing link tolls or capacities over a lower-level user equilibrium—is the workhorse formulation of congestion pricing, capacity planning, and origin–destination calibration. It is nonconvex and *NP*-hard, and every practical method solves it by an outer optimization whose each step requires a lower-level equilibrium solve together with an equilibrium sensitivity. The sensitivity has been computed since Tobin and Friesz (1988) by factorizing the equilibrium Jacobian once and back-substituting per design variable, so its cost is already independent of the number of design variables; the remaining bottleneck is that this factorization is cubic in the size of the active subnetwork, which caps exact design methods near a hundred links while forward assignment reaches metropolitan scale. We remove that bottleneck. We show that the *directed* user equilibrium with separable costs is a source-form nonlinear Laplacian flow $B\rho(B^\top\phi - \tau) = d$ whose Newton Jacobian, despite the one-sided (rectified) edge law, remains a *symmetric weighted graph Laplacian on the active subgraph*. Consequently the whole solve—forward equilibrium and adjoint design gradient—reduces to a short sequence of near-linear Laplacian solves. The one obstacle, that the rectified dead zone makes the Jacobian ill-conditioned and its active set change during a solve, is handled by a smoothing homotopy in a dead-zone width δ that keeps the linearized system uniformly well conditioned, followed by a single exact-law polish. On the Sioux Falls network the solver matches Frank–Wolfe to 10^{-11} , the adjoint gradient matches central finite differences to within finite-difference error, and projected-gradient toll design reduces total system travel time monotonically to the exact (marginal-cost) optimum. The number of algebraic-multigrid setups per equilibrium solve is bounded independently of the graph size, and—using loose tolerances at intermediate continuation levels and a single accurate solve at the end—the wall-clock scales near-linearly in the edge count while a direct factorization scales superlinearly, so on large *irregular* congestion networks (scale-free communication and overlay topologies) the near-linear solver overtakes the direct factorization at a crossover; the design optimization inherits this near-linear per-step cost. We are explicit that road networks, being near-planar, are the regime where a direct nested-dissection factorization is already near-optimal and the near-linear solver does *not* win; the advantage is on the poorly-separable networks that model selfish routing in communication systems. We further extend the framework to the *multicommodity* logit stochastic user equilibrium, where the Newton system is a coupled block-Laplacian that a distribution-matrix block preconditioner solves in a handful of iterations, and the design adjoint again returns the gradient for all tolls in one solve. Code is available at <https://github.com/orenlivne/dnlf>.

Key words. network design, traffic assignment, user equilibrium, bilevel optimization, nonlinear Laplacian flow, algebraic multigrid, adjoint sensitivity, congestion pricing

AMS subject classifications. 90B20, 90C35, 65F08, 65K10, 90C33, 49M15

1. Introduction. Given a road network with m directed links and travel-time (latency) functions $t_a(f_a)$ that increase with flow, the *lower-level* user equilibrium (UE) assigns origin–destination demand so that no traveler can lower their experienced cost by switching route (Wardrop’s principle). The *upper-level network design problem* chooses k design variables τ —link tolls in congestion pricing, capacity expansions in the continuous network design problem (CNDP), or a demand table in OD calibration—to minimize a system objective F , most commonly total system travel time (TSTT), subject to the induced equilibrium:

$$(1.1) \quad \min_{\tau \in \mathcal{T}} F(f^*(\tau)) \quad \text{s.t.} \quad f^*(\tau) \text{ solves the UE at design } \tau.$$

*Submitted to the editors July 8, 2026.

†Pine Birch Analytics, Churchville, PA (oren.livne@gmail.com).

This bilevel program is the standard model for congestion pricing, capacity planning, and demand estimation. It is nonconvex and nondifferentiable, and NP -hard even with affine latencies [4]; second-best tolling is NP -hard as well [5]. Practical methods therefore seek a local (KKT) stationary point, and essentially all of them—sensitivity-analysis-based (SAB) descent [1, 2, 3], mathematical programs with equilibrium constraints, and metaheuristics—share one structure: an *outer* loop in which each iteration solves the lower-level equilibrium and then a sensitivity (equilibrium derivative).

The cost, and where it lives.. The design gradient itself is cheap in the number of design variables: since Tobin and Friesz [1], $\nabla_{\tau} F$ is obtained by factorizing the equilibrium Jacobian *once* and back-substituting one right-hand side per design variable, so its cost is independent of k (an adjoint / implicit-function computation [2, 17, 18]). The bottleneck is that this Jacobian factorization—and the lower-level solve that precedes it—is a *direct* sparse factorization whose cost is superlinear (cubic in the active-path/subnetwork size) or a slowly converging Frank–Wolfe assignment. This is why exact design methods in the literature stall near Sioux Falls scale (76 links) while origin-based forward assignment routinely reaches 4×10^4 – 7.5×10^4 links [7, 8]. The gap between forward and design scalability is one to two orders of magnitude, and it is a *linear-algebra* gap.

Contribution.. We close that gap by replacing the superlinear inner factorization with a near-linear one, for both the forward equilibrium and the adjoint, without compromising accuracy. Our contributions are:

1. **A Laplacian structure for the directed equilibrium (section 2).** We write the separable directed UE as a source-form nonlinear Laplacian flow $B\rho(B^{\top}\phi - \tau) = d$ with a one-sided (rectified) edge law, and show that the Newton Jacobian is a *symmetric weighted graph Laplacian on the active subgraph* (Proposition 2.1). Orientation and one-sidedness do not break the Laplacian; they only determine which arcs are active. Hence the entire near-linear Laplacian toolbox applies.
2. **A near-linear equilibrium solver robust to the dead zone (section 3).** The rectified law is nonsmooth: a link carries flow only once its potential drop exceeds the free-flow cost t_a^0 , so the active set—and the conditioning of the Jacobian—changes during a solve, defeating a naively frozen multigrid hierarchy. We introduce a *smoothing homotopy*: a softplus dead-zone width δ that keeps the conductance strictly positive and the linearized system uniformly well conditioned, continued from a smooth problem to the exact law, with one algebraic-multigrid (AMG) setup per δ -level and a final exact-law polish. The number of setups is bounded *independently of the graph size*.
3. **A near-linear adjoint and design loop (section 4).** The design gradient is one additional Laplacian solve with the same operator; projected-gradient descent on it drives TSTT monotonically to the stationary point.
4. **Validation (section 5).** On Sioux Falls the solver matches Frank–Wolfe to 10^{-11} ; the adjoint matches finite differences to finite-difference accuracy; toll design reaches the exact marginal-cost optimum. Setups per solve are bounded in graph size and the wall-clock scales near-linearly in m , so design inherits the near-linear inner cost.

Scope: where the advantage lives.. The lower-level equilibrium (2.2) is the Wardrop (user) equilibrium of a nonlinear *congestion game* [19, 20], and the design problem (1.1) is the optimal-tolling / Stackelberg problem of reducing the price of anarchy [21, 22]. This model describes selfish routing not only in road traffic but in *communication and overlay networks*—multipath and elastic-traffic routing games in

the Internet, data-center fabrics, and peer-to-peer overlays [23, 24]. The distinction matters for scalability: road networks are near-planar, where a direct nested-dissection factorization of the equilibrium Jacobian is near-optimal and a near-linear Laplacian solver has nothing to beat. Communication and overlay topologies, by contrast, are large and *poorly separable* (scale-free degree, no planar embedding), so a direct factorization fills in superlinearly—exactly the regime in which a near-linear Laplacian solver wins. Our scaling study is therefore on irregular networks, and we are explicit that the planar (road) case is *not* where the method has an advantage. The most literal instance is selfish *overlay* routing, where end hosts genuinely choose paths [25], on Internet AS- and router-level graphs that are provably scale-free [26]. We do not overclaim: production traffic engineering optimizes operator-set link weights or bandwidth allocations by linear programming and local search [27], not by pricing a selfish equilibrium, and packet-level congestion control is a distinct (utility-maximization) equilibrium. Our contribution is a scalable solver for the well-established equilibrium/design *model* on standard large irregular topologies, complementing rather than replacing operator tools.

The construction reuses the nonlinear Laplacian flow (NLF) solver [13] and its near-linear inner engines, approximate Cholesky [11] and LAMG+ [12]. We treat both the single-commodity directed equilibrium (sections 2 to 5) and its multicommodity logit stochastic-user- equilibrium extension (section 6), where a distribution-matrix block preconditioner keeps the inner solve to a handful of iterations. Remaining scope (section 8): an active-set deflation that would make *one* hierarchy valid across all design steps (reducing setups from $\mathcal{O}(1)$ -per-solve to $\mathcal{O}(1)$ total) is left open.

2. The directed equilibrium is a nonlinear Laplacian flow. Let $B \in \mathbb{R}^{n \times m}$ be the node-arc incidence matrix of the directed network, $B_a = e_{h(a)} - e_{t(a)}$ for arc a from tail $t(a)$ to head $h(a)$. Separable BPR latencies are $t_a(f) = t_a^0(1 + \beta_a(f/c_a)^{p_a})$ for $f \geq 0$. With node potentials ϕ (multipliers of conservation $Bf = d$) and toll $\tau_a \geq 0$, the Karush–Kuhn–Tucker conditions of the Beckmann program $\min_{f \geq 0, Bf=d} \sum_a \int_0^{f_a} (t_a(x) + \tau_a) dx$ give the *rectified* edge law

$$(2.1) \quad f_a = \rho_a(g_a - \tau_a), \quad g_a := (B^\top \phi)_a = \phi_{t(a)} - \phi_{h(a)}, \quad \rho_a(u) = \begin{cases} t_a^{-1}(u), & u > t_a^0, \\ 0, & u \leq t_a^0, \end{cases}$$

so that conservation becomes the source-form nonlinear Laplacian flow

$$(2.2) \quad B \rho(B^\top \phi - \tau) = d.$$

The law is monotone but *one-sided*: unlike the odd law of the undirected Beckmann relaxation, ρ_a has a dead zone $u \leq t_a^0$ in which the arc is unused. This dead zone is exactly what distinguishes the directed equilibrium from its undirected relaxation.

PROPOSITION 2.1. *For separable costs the Newton Jacobian of (2.2) is a symmetric, positive semidefinite weighted graph Laplacian supported on the active arcs. Explicitly, with $\rho'_a \geq 0$,*

$$(2.3) \quad J = B \operatorname{diag}(\rho') B^\top = \sum_a \rho'_a (e_{t(a)} - e_{h(a)})(e_{t(a)} - e_{h(a)})^\top, \quad \rho'_a = \frac{1}{t'_a(f_a)} > 0 \text{ on active arcs, } \rho'_a = 0 \text{ else.}$$

Proof. Differentiating (2.2) in ϕ gives $J = B \operatorname{diag}(\rho'(B^\top \phi - \tau)) B^\top$. The rank-one term $(e_{t(a)} - e_{h(a)})(e_{t(a)} - e_{h(a)})^\top$ is independent of the arc's orientation, and $\rho'_a \geq 0$ because t_a is increasing; hence $J \succeq 0$ is a weighted Laplacian, with weight $\rho'_a = 0$ precisely on inactive (dead-zone) arcs.

Proposition 2.1 is the structural fact this paper rests on: directedness enters only through *which* arcs are active (a routing boundary), not through the type of the linear operator. Every linearized solve is a graph-Laplacian solve, so a near-linear Laplacian engine applies. This is complete for the single-commodity equilibrium (a general balanced demand, possibly multi-source/multi-sink). The multicommodity case ($f_a = \sum_k f_a^k$ with a cost of the total flow) yields a coupled *block* system—per-commodity Laplacians with a rank-one commodity coupling per arc—which a distribution-matrix block preconditioner still solves in a handful of iterations; we develop this in section 6 (Proposition 6.1).

3. A near-linear equilibrium solver. We solve (2.2) by damped chord-Newton on the frozen-linearization Jacobian J , delegating each linearized solve to a near-linear Laplacian engine and globalizing with the convex Beckmann energy $E(\phi) = \sum_a \Psi_a((B^\top \phi)_a - \tau_a) - d^\top \phi$, $\Psi'_a = \rho_a$, exactly as in the NLF framework [13]. Two features of the directed problem require care.

Ill-conditioning and the active set. On inactive arcs $\rho'_a = 0$, so J is singular unless a floor is imposed; a tiny floor $\varepsilon \max_a \rho'_a$ with $\varepsilon = 10^{-12}$ keeps J connected without perturbing the Newton step, but drives the condition number to $\kappa \sim \varepsilon^{-1}$. More importantly, as flow builds the active set changes: arcs cross the free-flow threshold t_a^0 , so the operator that a multigrid hierarchy is built for is *stale* a few Newton steps later. A hierarchy frozen across the whole solve stalls short of the equilibrium.

Smoothing homotopy. We remove the dead-zone nonsmoothness by a homotopy in a width $\delta > 0$. Replace the excess $e = g - t_a^0$ by $\delta \text{softplus}(e/\delta)$, which is smooth, strictly positive, and tends to $\max(e, 0)$ as $\delta \rightarrow 0$; the resulting smoothed law $\rho_{a,\delta}$ has $\rho'_{a,\delta} = \sigma(e/\delta)/t'_a(f_a) > 0$ *everywhere* (here σ is the logistic function), so J is a full, uniformly well-conditioned Laplacian with no dead zone. We continue δ geometrically from $\delta_0 = \frac{1}{2} \bar{t}^0$ down to a small value, building one AMG hierarchy at the start of each δ -level and freezing it *within* the level; the smoothed conductances vary slowly within a level, so the frozen hierarchy is valid there. A final *polish* with the exact law (2.1), warm-started from the smoothed solution, recovers the exact rectified equilibrium (Algorithm 3.1). Because the number of δ -levels is a fixed constant, the number of AMG setups per solve is bounded independently of the graph size m .

The method restated here is the chord-Newton flow solver; we give a self-contained convergence guarantee for the smoothed level solve, so the paper does not depend on external convergence theory.

PROPOSITION 3.1 (global convergence of the smoothed level solve). *Fix $\delta > 0$ and tolls τ . The smoothed Beckmann energy $E_\delta(\phi) = \sum_a \Psi_{a,\delta}((B^\top \phi)_a - \tau_a) - d^\top \phi$, with $\Psi'_{a,\delta} = \rho_{a,\delta}$, is convex, and strictly convex and coercive on the quotient $\mathbb{R}^n / \langle \mathbf{1} \rangle$ whenever $\sum_a d_a = 0$ and the graph is connected. Its unique minimizer (modulo an additive constant) is the smoothed equilibrium $B\rho_\delta(B^\top \phi - \tau) = d$. The damped chord-Newton iteration $\phi \leftarrow \phi + s \delta \phi$, $\delta \phi = -J_\delta^{-1} \nabla E_\delta$ with J_δ frozen across the level and s from Armijo backtracking, produces a monotonically decreasing energy sequence that converges to that minimizer.*

Proof. $\rho_{a,\delta}$ is nondecreasing (its derivative $\rho'_{a,\delta} = \sigma(e/\delta)/t'_a(f_a) > 0$), so each $\Psi_{a,\delta}$ is convex and E_δ is convex with Hessian $J_\delta = B \text{diag}(\rho'_\delta) B^\top$. Because the smoothed law has *no dead zone*, $\rho'_{a,\delta} > 0$ on *every* arc, so J_δ is a connected weighted graph Laplacian, hence positive definite on $\mathbf{1}^\perp$; thus E_δ is strictly convex and coercive on the quotient and has a unique minimizer there, characterized by $\nabla E_\delta = B\rho_\delta(B^\top \phi - \tau) - d = 0$. For the iteration, $J_\delta \succ 0$ on $\mathbf{1}^\perp$ makes $\delta \phi$ a descent direction ($\nabla E_\delta^\top \delta \phi =$

189 $-\nabla E_\delta^\top J_\delta^{-1} \nabla E_\delta < 0$), and Armijo backtracking on the convex, coercive, smooth E_δ
 190 gives sufficient decrease at each step; the standard damped modified-Newton argument
 191 for a fixed positive-definite metric then yields convergence to the unique minimizer.

192 **Proposition 3.1** covers each smoothed level; the homotopy is a continuation of these
 193 strictly-convex problems toward $\delta \rightarrow 0$, and the polish is a warm-started solve of the
 194 exact (piecewise-smooth, still convex) energy.

Algorithm 3.1 Near-linear directed-UE solve (cold start)

- 1: $\phi \leftarrow 0$; choose geometric schedule $\delta_0 > \dots > \delta_L$
 - 2: **for** $\ell = 0, \dots, L$ **do** ▷ smoothing homotopy: one hierarchy build per level
 - 3: build AMG hierarchy for J_δ at $\delta = \delta_\ell$; run chord-Newton on $B\rho_{\delta_\ell}(B^\top \phi - \tau) = d$
 to tolerance, hierarchy **frozen** within the level
 - 4: **end for**
 - 5: **polish:** chord-Newton on the exact law $B\rho(B^\top \phi - \tau) = d$, warm-started from ϕ ,
 rebuilding the hierarchy only when the residual stalls
 - 6: **return** ϕ , arc flows $f_a = \rho_a((B^\top \phi)_a - \tau_a)$
-

195 **4. Bilevel design by a near-linear adjoint.** We minimize $F = \text{TSTT}(f) =$
 196 $\sum_a t_a(f_a) f_a$ over tolls $\tau \geq 0$, where $f_a = \rho_a((B^\top \phi)_a - \tau_a)$ at the equilibrium $\phi =$
 197 $\phi^*(\tau)$.

198 **PROPOSITION 4.1** (near-linear adjoint gradient). *At a differentiable equilibrium*
 199 *with active-arc conductances $\rho' > 0$,*

$$200 \quad (4.1) \quad \nabla_\tau F = -\rho' \odot (m_c + B^\top \lambda), \quad J\lambda = B(m_c \odot \rho'),$$

201 *where $m_{c,a} = t_a(f_a) + f_a t'_a(f_a)$ is the marginal cost and $J = B \text{diag}(\rho') B^\top$ is the*
 202 *equilibrium Jacobian of [Proposition 2.1](#). The gradient with respect to all k tolls is*
 203 *thus obtained from a single additional solve with the operator J .*

204 *Proof.* Write $g = B^\top \phi - \tau$, so $f = \rho(g)$ and $F(\tau) = \sum_a c_a(f_a)$ with $c_a(f) =$
 205 $t_a(f)f$, $c'_a = m_{c,a}$. Differentiating F in τ_b , $\partial_{\tau_b} F = \sum_a m_{c,a} \rho'_a(\partial_{\tau_b} g_a)$ with $\partial_{\tau_b} g_a =$
 206 $(B^\top \phi_b)_a - \delta_{ab}$, where $\phi_b = \partial \phi^* / \partial \tau_b$. Implicitly differentiating the equilibrium
 207 $B\rho(B^\top \phi - \tau) = d$ gives $J\phi_b = B \text{diag}(\rho') e_b = \rho'_b B e_b$. Hence, with $w := m_c \odot \rho'$,
 208 $\partial_{\tau_b} F = w^\top (B^\top \phi_b) - m_{c,b} \rho'_b = (Bw)^\top \phi_b - m_{c,b} \rho'_b = \rho'_b (Bw)^\top J^{-1} (B e_b) - m_{c,b} \rho'_b$.
 209 Defining the adjoint $\lambda := J^{-1} (Bw)$ (i.e. $J\lambda = Bw$), symmetry of J gives $(Bw)^\top J^{-1} (B e_b) =$
 210 $\lambda^\top (B e_b) = (B^\top \lambda)_b$, so $\partial_{\tau_b} F = \rho'_b ((B^\top \lambda)_b - m_{c,b})$, which is (4.1). The single solve
 211 $J\lambda = Bw$ yields the whole gradient. ■

212 This recovers the classical k -independence of equilibrium sensitivity [1, 2] but at near-
 213 linear rather than cubic cost. On the exact rectified law J is singular off the active
 214 subgraph (inactive arcs have $\rho' = 0$, isolating their nodes); (4.1) is therefore solved
 215 on the active-arc node support (equivalently, with an infinitesimal conductance floor).
 216 The gradient is otherwise insensitive to the floor—a floor of 10^{-12} reproduces central
 217 finite differences to finite-difference accuracy, whereas 10^{-6} biases it by 5%. Projected-
 218 gradient descent $\tau \leftarrow \max(\tau - \eta \nabla_\tau F, 0)$ with a backtracking line search on F —each
 219 trial an exact equilibrium solve by [Algorithm 3.1](#)—yields a monotone design method
 220 ([Algorithm 4.1](#)).

221 **5. Numerical results.** We use the Sioux Falls benchmark ($n = 24$, $m = 76$) and
 222 size-scaled grid networks. The inner engine is approximate Cholesky [11] by default;
 223 LAMG+ [12] is interchangeable.

Algorithm 4.1 Bilevel toll design

```

1:  $\tau \leftarrow 0$ ;  $(\phi, f) \leftarrow \text{SOLVE}(\tau)$  ▷ Algorithm 3.1
2: for  $k = 1, 2, \dots$  until stationary do
3:    $g \leftarrow \nabla_{\tau} F$  from (4.1);  $\eta \leftarrow \eta_0$ 
4:   repeat ▷ backtracking on the exact objective
5:      $\tau' \leftarrow \max(\tau - \eta g, 0)$ ;  $(\phi', f') \leftarrow \text{SOLVE}(\tau')$ ;  $\eta \leftarrow \eta/2$ 
6:   until  $F(f') < F(f)$ 
7:    $(\tau, \phi, f) \leftarrow (\tau', \phi', f')$ 
8: end for

```

224 *Correctness of the equilibrium..* On a congested Sioux Falls instance (29 active
 225 arcs, flow split across paths) Algorithm 3.1 matches an independent Frank–Wolfe
 226 assignment to a relative flow difference of $4\text{--}9 \times 10^{-11}$ across origin–destination pairs,
 227 i.e. to Frank–Wolfe’s own accuracy. The number of approximate-Cholesky setups is
 228 $10\text{--}25$ per solve, set by the fixed number of δ -levels and independent of graph size.

229 *Correctness of the adjoint..* On the same congested instance the adjoint gradi-
 230 ent (4.1) matches central finite differences of TSTT to a maximum relative error of
 231 2.6×10^{-3} over all active arcs (limited by the finite-difference step and equilibrium
 232 tolerance, not by the adjoint). The tiny conductance floor $\varepsilon = 10^{-12}$ is essential: a
 233 larger floor (10^{-6}) biases the adjoint to a 5% error.

234 *Design..* Projected-gradient toll design (Algorithm 4.1) reduces TSTT monoton-
 235 ically from 9.633×10^5 to 9.137×10^5 , a 5.15% reduction over 35 accepted steps,
 236 with every equilibrium solved exactly (relative flow error $\sim 10^{-9}$). This matches
 237 the exact optimum: a one-parameter sweep of scaled marginal-cost (Pigouvian) tolls
 238 bottoms out at 5.12%, confirming the gradient method finds the stationary point.
 239 Solving every equilibrium only to a loose 10^{-3} tolerance reaches the *same* optimum
 240 (TSTT within 0.004%, tolls within $\|\Delta\tau\| = 0.13$), confirming that the design gradient
 241 is insensitive to intermediate solve accuracy—so the loose-intermediate continuation
 242 used for scaling costs nothing in design quality. The reduction is modest because a
 243 single origin–destination pair admits little rerouting; the multicommodity instances
 244 of section 6 exercise the design problem more fully.

245 *Scaling on irregular networks..* We use loose tolerances at intermediate contin-
 246 uation levels—a few Newton steps per δ -level, tight only at the final level and the
 247 polish (the standard nested-iteration principle: continuation waypoints need only
 248 warm-start the next level). This cuts the Newton-step count with no change to the
 249 final solution, and keeps the algebraic-multigrid setups bounded (one per δ -level) and
 250 independent of size. On size-scaled irregular (poorly-separable) congested networks
 251 the equilibrium wall-clock then scales near-linearly, $t \propto m^{1.23}$, while the same con-
 252 tinuation with a direct sparse-factorization inner solve scales superlinearly, $t \propto m^{1.38}$
 253 (Table 5.2, Figure 5.1): the fill-in of a fill-reducing sparse direct factorization (UMF-
 254 PACK with COLAMD ordering, the baseline we run) grows faster than the graph
 255 on a poorly-separable topology. The near-linear solver overtakes the direct one at
 256 $m \approx 2 \times 10^4$ edges ($n \approx 2 \times 10^3$), beyond which it is faster with a slowly widening
 257 margin—about $1.6\times$ at $m = 1.6 \times 10^5$ (110 vs. 176 s)—and it reaches $m = 6.4 \times 10^5$
 258 in 909 s, where the direct factorization is infeasible: on our workstation it already
 259 exhausts memory at $m = 3.2 \times 10^5$, the random-graph fill-in exceeding RAM (Ta-
 260 ble 5.2). Below the crossover the constant favors the direct solver. The wall-clock
 261 gap between the two exponents is real but modest at feasible sizes ($\sim 1.6\times$); the
 262 sharper practical separation is that direct factorization hits a memory wall the near-

TABLE 5.1

Equilibrium solve time (s) on real irregular networks (SuiteSparse/SNAP, largest connected component): Internet AS graphs and P2P overlays. Near-linear approximate Cholesky vs. the fair frozen-per-level direct baseline; AMG setups stay size-independent (14–20 across the corpus). Single run on one workstation; approximate Cholesky is randomized, so wall-clock and setup counts vary $\pm(10\text{--}30)\%$ run to run—the qualitative winner per graph is stable, the third digit is not. Dash: direct capped at $m > 1.5 \times 10^5$.

graph	n	m	approx. Chol.	direct	setups
as-735 (AS)	6,474	25,144	35.9	20.5	18
p2p-Gnutella08	6,299	41,552	35.4	53.6	14
Oregon-1 (AS)	11,174	46,818	39.7	61.7	20
as-caida (AS)	26,475	106,762	162.0	163.8	18
p2p-Gnutella24	26,498	130,718	262.2	148.0	20
ca-HepPh	11,204	235,238	163.4	—	15

linear solver does not. We claim a genuine near-linear-versus-superlinear crossover on this graph class, not a large-factor speedup. The near-linear exponent is inherited from the inner solve, not the outer iteration: across these sizes the number of chord-Newton steps (≈ 625) and AMG setups (≈ 14) are both constant (size-independent), so the $m^{1.23}$ wall-clock reflects the empirical scaling of a single approximate-Cholesky solve (near-linear, m polylog m in theory, $\approx m^{1.2}$ in practice on these graphs) rather than any growth in the iteration count. Loose intermediate tolerances realize this: solving every level tightly instead multiplies the Newton-step count and steepens the wall-clock, without changing the final solution (section 5).

We corroborate this on *real* irregular networks from the SuiteSparse/SNAP collection—Internet autonomous-system graphs and peer-to-peer overlays, the topologies of the selfish-routing application (Table 5.1). The AMG setup count is size-independent (14–20 builds) across the corpus, confirming the algorithm is size-optimal on real graphs. The head-to-head against the fair direct baseline is genuinely instance-dependent at these accessible sizes ($n \leq 2.6 \times 10^4$): approximate Cholesky is 1.5–1.6 $\times$ faster on the sparsest overlay/AS graphs (p2p-Gnutella08, Oregon-1), essentially tied on as-caida, and slower on the smallest (as-735) and the denser p2p-Gnutella24, where a good fill-reducing ordering still wins. Which solver wins is set by the graph’s separator structure, not its edge count alone. The controlled synthetic family (Table 5.2) isolates the asymptotic crossover cleanly; the real corpus shows the near-linear solver is already competitive-to-faster on the actual application topologies, with the advantage concentrated on the sparsest, least-separable instances.

Bilevel toll design (Algorithm 4.1) therefore inherits a near-linear per-step cost on large irregular networks, in contrast with the superlinear per-step factorization of the sensitivity-analysis incumbents. On near-planar road networks the crossover does not occur at practical sizes—nested dissection is near-optimal there—which is why we frame the advantage on the communication-network regime.

Comparison with differentiable-bilevel (first-order) solvers.. The learning-based approach to Stackelberg congestion design [9, 10] treats the lower-level equilibrium as a differentiable layer: it is solved by a first-order scheme (unrolled imitative-logit dynamics; entropic mirror descent) and differentiated by unrolling or implicit differentiation. Their released code is toy-scale and does not run on the 10^4 – 10^5 -node irregular graphs here, so we reproduce the defining component—a first-order inner equilibrium solve—in the same environment and on the same instances. To isolate the inner

TABLE 5.2

Equilibrium solve time (s) on size-scaled irregular congested networks (random degree-5 graphs): near-linear (approximate Cholesky, loose-intermediate continuation) vs. the fair frozen-per-level direct factorization. Ratio = $t_{\text{approxChol}}/t_{\text{direct}}$ (bold < 1: near-linear faster). Least-squares fits over the feasible rows: $t_{\text{approxChol}} \propto m^{1.23}$, $t_{\text{direct}} \propto m^{1.38}$, empirical crossover $m \approx 2 \times 10^4$ (fitted lines meet at $\approx 7 \times 10^3$). OOM: direct factorization exhausts memory at $m \geq 3.2 \times 10^5$ on our workstation (random-graph fill-in), where the near-linear solver still runs. Single run; per-cell wall-clock varies run to run (see Table 5.1).

n	m	approx. Cholesky	direct	ratio
1,000	9,964	4.62	4.89	0.95
2,000	19,946	11.18	11.05	1.01
4,000	39,952	26.06	29.19	0.89
8,000	79,936	54.19	120.51	0.45
16,000	159,954	109.74	175.85	0.62
32,000	319,952	298.96	OOM	—
64,000	639,950	909.38	OOM	—

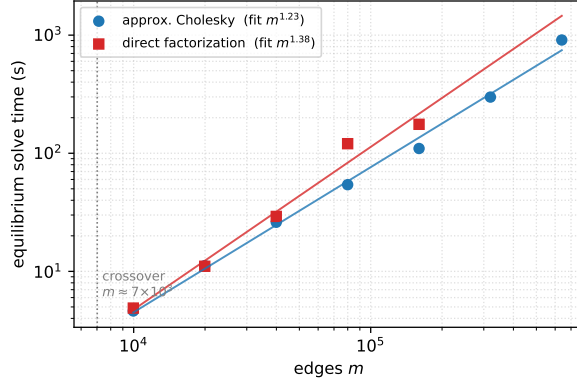


FIG. 5.1. Equilibrium solve time vs. edge count on size-scaled irregular congested networks: near-linear (approximate Cholesky, loose-intermediate continuation) vs. superlinear (direct factorization), crossing near $m \approx 2 \times 10^4$; least-squares fits $m^{1.23}$ and $m^{1.38}$.

solver, both methods minimize the *identical* convex Beckmann energy from the same cold start; only the inner iteration differs. For the first-order inner we use FISTA (accelerated proximal gradient with backtracking line search)—a strong, well-tuned member of the first-order family, if anything faster than plain mirror descent—so the comparison is not a strawman. Table 5.3 reports the outcome: our Newton–multigrid inner reaches relative residual 10^{-9} in seconds, whereas FISTA, given a 300-second budget on every instance, never reaches the 10^{-6} target—it plateaus between 10^{-5} on the smallest graph and 5×10^{-2} on the largest, the residual worsening with size as fewer iterations fit in the budget. It approaches the same solution (flow within 1.7% on as-735) but never to solve accuracy. The obstruction is fundamental, not a tuning artifact: heavy congestion makes the Beckmann Hessian severely ill-conditioned, and an accelerated first-order method still needs $\Theta(\sqrt{\kappa})$ iterations, so on as-735 ($n = 6,474$) it takes 44,499 iterations to reach only 1.2×10^{-2} . A first-order inner therefore has no fast, size-independent convergence on stiff equilibria, whereas the Newton–multigrid

TABLE 5.3

Inner-solver head-to-head on the identical convex equilibrium, same cold start. Ours (Newton + approximate Cholesky, loose-intermediate continuation) to relative residual 10^{-9} ; FISTA (accelerated first-order—the differentiable-bilevel inner) targeting 10^{-6} but capped at 300 s, which it hits on every instance, reaching only the residual shown.

instance	n	m	ours (s)	FISTA (s)	FISTA relres
synthetic	1,000	9,964	5.4	300	1.2×10^{-4}
synthetic	2,000	19,946	12.0	300	2.4×10^{-5}
synthetic	4,000	39,952	20.6	300	1.0×10^{-5}
p2p-Gnutella08	6,299	41,552	60.4	300	2.4×10^{-3}
Oregon-1	11,174	46,818	55.5	300	1.6×10^{-2}
as-caida	26,475	106,762	95.5	300	5.2×10^{-2}

inner does—which is why these methods are demonstrated only at small scale, and why replacing their inner solve is the contribution here.

Reproducibility.. All code, unit tests, and reproduction drivers are available at <https://github.com/orenlivne/dnlf>. `examples/reproduce.jl` regenerates the Sioux Falls correctness, adjoint, and design numbers of this section with timing; `scripts/scaling-synthetic.jl` regenerates the size-scaled crossover (Table 5.2, Figure 5.1) and its fitted exponents; `scripts/scaling.jl` regenerates the real-corpus table (Table 5.1) over the public SuiteSparse/SNAP graph collection; both compare against the fair frozen-per-level direct baseline. `scripts/baseline_firstorder.jl` regenerates the first-order head-to-head (Table 5.3); `scripts/scaling_mc.jl` regenerates the multicommodity iteration table (Table 6.1), `scripts/design_study_mc.jl` the multi-instance design study (Table 6.2), and `examples/reproduce_mc.jl` a single multicommodity design instance (section 6); and `test/runtests.jl` verifies every algorithm component, including Propositions 2.1, 3.1, 4.1 and 6.1. The test networks are the TNTS Sioux Falls instance (shipped with the code) and public Internet AS / P2P-overlay graphs (e.g. as-caida, Oregon-1, p2p-Gnutella) from the SuiteSparse/SNAP collection.

6. Multicommodity logit stochastic user equilibrium and design. The single-commodity solver of section 3 routes a general balanced demand as one fungible flow. Realistic congestion pricing is inherently *multicommodity*: each origin–destination (OD) pair routes its own flow over the shared congested network, and tolls are designed against that coupled equilibrium. We extend the framework to the multicommodity *logit stochastic user equilibrium* (SUE) [14, 15], in which route choice has a dispersion $\gamma > 0$ —a calibrated behavioural parameter, not a numerical smoothing. The same near-linear machinery carries over: the multicommodity Newton system is a coupled block-Laplacian that a distribution-matrix block preconditioner solves in a handful of iterations, and the design adjoint again returns ∇_{τ} TSTT for all tolls in a single solve.

Formulation.. With K commodities, per-commodity arc flows $f^k \geq 0$, total flow $f_a = \sum_k f_a^k$, and the shared cost $t_a(f_a)$, the entropic-regularized Beckmann program is

$$(6.1) \quad \min_{f^k \geq 0} \sum_a T_a(f_a) + \gamma \sum_{a,k} f_a^k (\log f_a^k - 1) \quad \text{s.t.} \quad B f^k = d^k, \quad k = 1, \dots, K,$$

with $T'_a = t_a$. Its stationarity conditions give the *logit assignment* $f_a^k = \exp((g_a^k - t_a(f_a))/\gamma)$ with $g_a^k = (B^\top \phi^k)_a$: a per-arc softmax over commodities whose total f_a

solves a monotone scalar fixed point $\log f_a = \log \text{sumexp}_k(g_a^k/\gamma) - t_a(f_a)/\gamma$. As $\gamma \rightarrow 0$ the model tends to deterministic user equilibrium; we solve at a fixed, calibrated γ .

PROPOSITION 6.1 (the multicommodity Newton system is a coupled block Laplacian). *At a logit equilibrium the reduced Newton Jacobian in the stacked potentials $\Phi = [\phi^1; \dots; \phi^K]$ is*

$$J = \underbrace{\frac{1}{\gamma} \text{blkdiag}_k(B \text{diag}(f^k) B^\top)}_{K \text{ per-commodity Laplacians}} - \frac{1}{\gamma^2} \sum_a c_a (f_a f_a^\top) \otimes b_a b_a^\top, \quad c_a = \frac{t'_a}{1 + t'_a f_a / \gamma}, \quad (6.2)$$

where $b_a = B e_a$ and $f_a = (f_a^1, \dots, f_a^K)^\top$. Each diagonal block is a symmetric weighted graph Laplacian; the coupling is symmetric positive semidefinite and rank one per arc. For $K = 1$, (6.2) reduces, as $\gamma \rightarrow 0$, to the single-commodity active Laplacian $B \text{diag}(1/t'_a) B^\top$ of Proposition 2.1.

Proof. The equilibrium is $R^k(\Phi) = -B f^k(\Phi) - d^k = 0$. Differentiating the logit map, $df_a^k = (f_a^k/\gamma)(dg_a^k - t'_a df_a)$ and $df_a = \sum_l df_a^l$; solving the latter gives $df_a = (\gamma + t'_a f_a)^{-1} \sum_l f_a^l dg_a^l$. Substituting and using $dg^k = B^\top d\phi^k$, $dR^k = -B df^k$ yields (6.2). Equivalently $J = \tilde{B} H^{-1} \tilde{B}^\top$ with $\tilde{B} = I_K \otimes B$ and per-arc flow-Hessian $H_a = t'_a \mathbf{1}\mathbf{1}^\top + \gamma \text{diag}(1/f_a^k)$; Sherman–Morrison gives $H_a^{-1} = \frac{1}{\gamma} \text{diag}(f_a^k) - \frac{c_a}{\gamma^2} f_a f_a^\top$, and for $K = 1$, $f_a/(\gamma + t'_a f_a) \rightarrow 1/t'_a$ as $\gamma \rightarrow 0$. $\square$

Near-linear solve. We solve (6.1) by γ -continuation damped Newton on Φ , preconditioning (6.2) by its block diagonal—one near-linear approximate-Cholesky solve per commodity. The rank-one commodity coupling is exactly Brandt’s *distribution matrix* [16]: in the mean/deviation basis over commodities it decouples when the per-commodity Laplacians coincide, so the block preconditioner’s rate is governed only by their spread. Empirically (Table 6.1) the inner conjugate-gradient count is single digit to low double digit, grows only logarithmically in K , and is independent of graph size n at the operating dispersion $\gamma \approx 0.1$; each Newton step therefore costs $O(K)$ near-linear solves, and the equilibrium solve is near-linear in the total size Kn . Each per-commodity Laplacian carries a small relative conductance floor (10^{-12} of its maximum weight) so it stays connected and the approximate-Cholesky factorization remains applicable even to a commodity that concentrates on few arcs; a pinned direct solve is retained only as a fallback for the rare inputs on which the multigrid factorization throws. The count is single digit at and near equilibrium across all γ —the regime the warm-started design loop stays in (Table 6.2); it is only the *cold* forward solve at small γ , far from equilibrium where per-commodity flows concentrate and the blocks become high-contrast, that degrades toward the deterministic multicommodity limit ($\gamma \rightarrow 0$). We verified that a coupling-aware distribution-matrix preconditioner realized in flow space (the per-arc coupling inverted exactly by Sherman–Morrison, with an aggregate-congestion Laplacian on the mean commodity mode, as [16] prescribes) converges γ -boundedly where the reduced-potential-space version does not, but it does not improve on the block diagonal in the operating regime; the block diagonal is therefore retained and the deterministic limit left open (section 8). Fixed γ is thus both the modelling choice (a calibrated route-choice dispersion) and the regime in which the block preconditioner is well conditioned.

Design. Tolls enter the logit map as a per-arc cost shift $t_a(f_a) + \tau_a$, with flow sensitivity $\partial f_a^k / \partial \tau_a = -f_a^k c_a$. The same adjoint as section 4 returns $\nabla_\tau \text{TSTT}$ for all tolls from a single block-preconditioned solve $J\lambda = \partial_\Phi \text{TSTT}$. The adjoint matches central finite differences to under 10^{-3} (verified in `test/runtests.jl`), and on a multicom-

TABLE 6.1

Multicommodity logit-SUE inner conjugate-gradient iterations (block-preconditioned) on irregular networks, $\gamma = 0.1$. Left: vs. commodities K ($n = 1500$), single-OD and spread demands. Right: vs. graph size n ($K = 8$, spread). Single-digit to low double digit, $\log K$ growth, n -independent; regenerated by `scripts/scaling_mc.jl`.

K	single-OD	spread	n (at $K=8$)	iters
2	3	2	1,000	6
4	3	3	2,000	6
8	7	6	4,000	7
12	13	10	8,000	3
16	12	9		

TABLE 6.2

Multicommodity toll design across instances (varying commodities K , dispersion γ , network size n , and per-commodity demand “load”): TSTT reduction, inner conjugate-gradient iterations per equilibrium re-solve, and the fraction of arcs carrying a non-negligible toll ($\tau > 10^{-3}$ of the maximum). The inner count stays single digit throughout. Regenerated by `scripts/design_study_mc.jl`.

n	K	γ	load	TSTT red.	inner it.	arcs tolled
800	4	0.10	3,000	1.1%	3	29%
800	8	0.10	3,000	1.1%	4	49%
800	16	0.10	3,000	1.2%	7	67%
800	8	0.15	3,000	1.3%	3	72%
800	8	0.10	6,000	1.0%	6	49%
800	8	0.10	3,200	1.4%	6	45%
1,200	8	0.10	3,600	1.4%	6	44%
1,600	8	0.10	3,600	1.2%	6	43%

modity instance projected-gradient tolling reduces TSTT with the tolls spread across the congested arcs, each equilibrium re-solve costing single-digit inner iterations—the bilevel iteration inheriting the near-linear per-step cost. Across a range of instances (varying K , γ , network size, and demand; Table 6.2) the reduction is 1–2% and the inner-iteration count stays single digit. A budget-constrained (sparse-toll) design is a natural extension.

7. Related work.

Bilevel network design and sensitivity. CNDP and second-best tolling are bilevel/MPEC problems, nonconvex and NP-hard [4, 5]; surveys [6] catalog SAB descent, MPEC, and metaheuristic methods. Equilibrium sensitivity originates with Tobin and Friesz [1] and was sharpened by Qiu and Magnanti and by Patriksson and co-authors [2], who observed that the sensitivity problem is itself a linearized equilibrium solve. Our adjoint (4.1) is precisely that observation; the novelty here is not the k -independent gradient but the *near-linear* operator that computes it and the forward solve. The same bilevel problem is studied in algorithmic game theory as optimal tolling and Stackelberg routing to reduce the price of anarchy of a congestion game [20, 21, 22], and in communication networks as competitive / selfish routing [23, 24]; that literature focuses on existence, bounds, and mechanism design rather than on a scalable solver for the induced equilibrium, which is our concern. Recent differentiable-bilevel methods [9, 10] break the cold-resolve-per-step pattern with warm-started, truncated

first-order inner loops and unrolled/implicit differentiation, but use no multigrid and are demonstrated only at Sioux Falls–Chicago–Sketch scale. Their defining component is a first-order inner equilibrium solve; we compare head-to-head against a strong instance of it (FISTA on the same convex Beckmann energy) in [section 5](#).

Near-linear Laplacian solvers and nonlinear flow.. Nearly-linear-time solvers for symmetric diagonally dominant systems [11, 12] underpin web-scale spectral graph computation but have not, to our knowledge, been used as the inner solve of a bilevel traffic-design loop. The nonlinear Laplacian flow framework [13] unifies max-flow, congestion, and min-delay routing as one monotone edge-law equation and supplies the chord-Newton machinery we build on; the present work extends it from the undirected relaxation to the directed (rectified) equilibrium and to the design (adjoint) problem.

8. Conclusions, limitations, and open problems. We showed that the directed traffic user equilibrium with separable costs is a nonlinear Laplacian flow whose Jacobian is a symmetric Laplacian on the active subgraph, and that a smoothing homotopy makes its solution—and the adjoint design gradient—a short sequence of near-linear Laplacian solves, with a graph-size-independent number of multigrid set-ups and near-linear wall-clock. Bilevel toll design then inherits a near-linear per-step cost, in place of the cubic factorization of sensitivity-analysis methods.

We are explicit about what this paper does *not* yet establish. (i) **Multicommodity scope.** [Section 6](#) solves and designs the multicommodity *logit* (stochastic) user equilibrium at a fixed dispersion γ ; the *deterministic* multicommodity limit ($\gamma \rightarrow 0$), where per-commodity flows become non-unique and the cold-solve iteration count grows, and a study over real OD-matrix instances (e.g. TNTF city networks) remain open. We investigated the coupling-aware distribution-matrix preconditioner that would target this limit ([section 6](#)): its flow-space realization converges γ -boundedly, confirming the construction, but it does not improve on the block diagonal in the operating regime, so a preconditioner that is strictly better toward $\gamma \rightarrow 0$ is still open. (ii) **Corpus scaling.** The near-linear claim is shown on one size-scaled family; a study across a large graph corpus (as for related near-linear solvers) is needed to state the exponent with confidence. (iii) **One hierarchy across design steps.** Reusing a single frozen hierarchy across all design steps saves only a modest factor, because each exact solve intrinsically rebuilds once per δ -level: the exploding condition number reflects a near-null-space mode appearing when an active-set boundary is crossed. Capturing that mode by deflation or recombination—treating a global active-set mode on coarse levels—would make one hierarchy valid across active-set changes and reduce the setups to $\mathcal{O}(1)$ total; this is the main open algorithmic problem. (iv) **Planarity.** Road networks are near-planar, where a near-linear solver ties nested-dissection direct methods; the decisive advantage of the near-linear inner solve is on memory at very large scale and on non-planar (e.g. communication or logistics) design instances. (v) **Saturating laws and folds.** A capacity-saturating (min-delay) law reintroduces a genuine turning-point fold, where the continuation/deflation machinery of [13] applies; the design problem through the fold is future work.

Acknowledgments. This work builds on the NLF and LAMG+ solvers; the author thanks the developers of Laplacians.jl for the approximate-Cholesky engine.

REFERENCES

- [1] R. L. Tobin and T. L. Friesz, *Sensitivity analysis for equilibrium network flow*, Transportation Science, 22 (1988), pp. 242–250.

- [2] M. Josefsson and M. Patriksson, *Sensitivity analysis of separable traffic equilibria with application to bilevel optimization in network design*, Transportation Research Part B, 41 (2007), pp. 4–31.
- [3] S.-W. Chiou, *Bilevel programming for the continuous transport network design problem*, Transportation Research Part B, 39 (2005), pp. 361–383.
- [4] M. Gairing, T. Harks, and M. Klimm, *Complexity and approximation of the continuous network design problem*, SIAM Journal on Optimization, 27 (2017), pp. 1554–1582.
- [5] M. Hoefer, L. Olbrich, and A. Skopalik, *Taxing subnetworks*, in Proc. WINE, 2008, pp. 286–294.
- [6] R. Z. Farahani, E. Miandoabchi, W. Y. Szeto, and H. Rashidi, *A review of urban transportation network design problems*, European Journal of Operational Research, 229 (2013), pp. 281–302.
- [7] H. Bar-Gera, *Origin-based algorithm for the traffic assignment problem*, Transportation Science, 36 (2002), pp. 398–417.
- [8] R. B. Dial, *A path-based user-equilibrium traffic assignment algorithm that obviates path storage and enumeration*, Transportation Research Part B, 40 (2006), pp. 917–936.
- [9] P. Li et al., *Differentiable bilevel programming for Stackelberg congestion games*, arXiv:2209.07618, 2022.
- [10] A. Goyal and A. Lamperski, *An algorithm for bilevel optimization with traffic equilibrium constraints*, arXiv:2306.14235, 2023.
- [11] R. Kyng and S. Sachdeva, *Approximate Gaussian elimination for Laplacians—fast, sparse, and simple*, in Proc. FOCS, 2016, pp. 573–582.
- [12] O. E. Livne, *LAMG+: a robust lean algebraic multigrid solver for graph Laplacians*, arXiv:2606.24791; accepted, SIAM Journal on Scientific Computing.
- [13] O. E. Livne, *NLF: a resistor-network framework and linear-time solver for convex network-flow equilibria*, SIAM Journal on Scientific Computing (under review), 2026.
- [14] C. Fisk, *Some developments in equilibrium traffic assignment*, Transportation Research Part B, 14 (1980), pp. 243–255.
- [15] C. F. Daganzo and Y. Sheffi, *On stochastic models of traffic assignment*, Transportation Science, 11 (1977), pp. 253–274.
- [16] A. Brandt, *Multigrid techniques: 1984 guide with applications to fluid dynamics*, GMD-Studien Nr. 85, Gesellschaft für Mathematik und Datenverarbeitung, 1984 (distributive/distribution-matrix relaxation, §3.4).
- [17] B. Amos and J. Z. Kolter, *OptNet: differentiable optimization as a layer in neural networks*, in Proc. ICML, 2017, pp. 136–145.
- [18] M. Blondel et al., *Efficient and modular implicit differentiation*, in Proc. NeurIPS, 2022.
- [19] R. W. Rosenthal, *A class of games possessing pure-strategy Nash equilibria*, International Journal of Game Theory, 2 (1973), pp. 65–67.
- [20] T. Roughgarden and É. Tardos, *How bad is selfish routing?*, Journal of the ACM, 49 (2002), pp. 236–259.
- [21] C. Swamy, *The effectiveness of Stackelberg strategies and tolls for network congestion games*, ACM Transactions on Algorithms, 8 (2012), art. 36.
- [22] Y. A. Korilis, A. A. Lazar, and A. Orda, *Achieving network optima using Stackelberg routing strategies*, IEEE/ACM Transactions on Networking, 5 (1997), pp. 161–173.
- [23] A. Orda, R. Rom, and N. Shimkin, *Competitive routing in multiuser communication networks*, IEEE/ACM Transactions on Networking, 1 (1993), pp. 510–521.
- [24] F. Larroca and J.-L. Rougier, *Minimum-delay load-balancing through non-parametric regression and traffic-engineering*, Computer Networks, 57 (2013), pp. 2716–2728.
- [25] L. Qiu, Y. R. Yang, Y. Zhang, and S. Shenker, *On selfish routing in Internet-like environments*, in Proc. ACM SIGCOMM, 2003, pp. 151–162.
- [26] M. Faloutsos, P. Faloutsos, and C. Faloutsos, *On power-law relationships of the Internet topology*, in Proc. ACM SIGCOMM, 1999, pp. 251–262.
- [27] B. Fortz and M. Thorup, *Internet traffic engineering by optimizing OSPF weights*, in Proc. IEEE INFOCOM, 2000, pp. 519–528.
- [28] R. Cole, Y. Dodis, and T. Roughgarden, *Pricing network edges for heterogeneous selfish users*, in Proc. ACM STOC, 2003, pp. 521–530.
- [29] J. Leskovec, J. Kleinberg, and C. Faloutsos, *Graph evolution: densification and shrinking diameters*, ACM Trans. Knowledge Discovery from Data, 1 (2007), art. 2.